

Test Case Lab Excellence — the Measuring Stick

Branch: `feat/generation-quality-engine`

Status: Phase 1 (Golden Benchmark) + Phase 2 (Self-Grading Scorer) — shipped, tested, committed.

Scope discipline: This turn built only the yardstick. No coverage categories were “fixed”, no CI gate was wired, and no new intelligence engine was added. Those are Phases 3–5 and start only after the benchmark tells us where the biggest gap is.

Why this exists

We stop selling “we generated 120 test cases.” We start selling measurable QA quality:

“Your requirement achieved 91% QA coverage against our Senior QA Architect Benchmark.”

To say that honestly we need two things that did not exist before:

1. **A gold standard** — what a Senior QA Architect expects a suite to cover, authored by hand, sealed, and never bent toward whatever the generator happens to produce.
2. **A scorer** — a function that grades any generated suite against that gold standard and returns one comparable number per requirement, plus a category breakdown that tells us exactly what is missing.

Both now exist.

Phase 1 — The Golden Benchmark

`scripts/gold-benchmarks.ts` was rewritten to be **category-first** and expanded from 5 to **10 hand-authored requirements**:

Create Employee · Login · Forgot Password · Checkout · Leave Request · Registration · Edit Employee · Shopping Cart · Payment · User Profile.

Every expectation is tagged with **one canonical QA category** from a fixed, shared taxonomy so the leaderboard axis is identical across all requirements:

Category	What it captures
Functional	The happy paths the feature exists to deliver
Validation	Field-level input checks (mandatory, format, mismatch)
Business Rule	Domain rules (uniqueness, balances, pricing, approval, immutability, idempotency, stock)
Negative	Failure / error paths (declined, wrong credential, rejected token, non-existent record)
Boundary	Limits & odd-but-legal inputs (min/max length, whitespace, special chars, timeouts, concurrency)
Security	Confidentiality / abuse defence (hashing, enumeration, injection, rate limiting, PCI, session)
Authorization	Who may perform the action (role, ownership, permission)
File Upload	File inputs (valid, wrong format, oversized, corrupt)
Search	Retrievability after the operation (by id, by name)
Navigation	Redirects, confirmations, notifications, cancel, logout
Data Integrity	Persistence & consistency (saved, decremented, deducted, audit trail, no partial writes)

Two design rules make it trustworthy:

- **The benchmark is sealed.** Expectations were authored from the requirement text + senior-QA expertise alone, never from generator output. If a concept is genuinely absent from a suite, it must score as missing. We never move an expectation toward the generator to make a number look better.
- **The taxonomy is fixed and shared**, so “Boundary” means the same thing for Login as it does for Payment, and per-category scores are comparable across requirements. (The founder’s “Permission” concept is folded into **Authorization**; documented in the file header.)

The 5 original benchmarks kept their exact match phrases and requirement text — they were only re-tagged by category — so the existing loss-audit harness numbers do not shift.

Phase 2 — The Scorer (self-grading)

`scripts/qa-architect-scorer.ts` has two layers:

1. `scoreBenchmark(benchmark, generatedCases)` — a pure function

Given a benchmark and the text of a generated suite, it returns:

- **per-category coverage %** (covered ÷ expected in that category),
- **overall %** (covered ÷ all expectations),
- **weight-aware %** — a miss on a critical expectation costs 3x, high 2x, medium 1x, so a suite that covers the trivia but drops the critical duplicate-ID check is scored honestly,
- the exact **missing list**, worst-weighted first.

It has no I/O and does not care where the cases came from — live LLM, planner, or a paste-in. This is the unit-tested heart (`tests/unit/qa-architect-scorer.test.ts` , **5/5 passing**, deterministic, no planner/model dependency).

2. `buildLeaderboard(scores)` — per-requirement score + a single average

The number that goes up as the generator improves. One KPI per release.

One shared matcher

The token-aware coverage detector (stemming + token-boundary matching, so `search` → `searchable` matches but `all` does **not** match inside `manually`) was extracted into `scripts/coverage-match.ts` and is now imported by **both** the scorer and the existing audit harness — one rule, no fragmentation, no drift between the two tools.

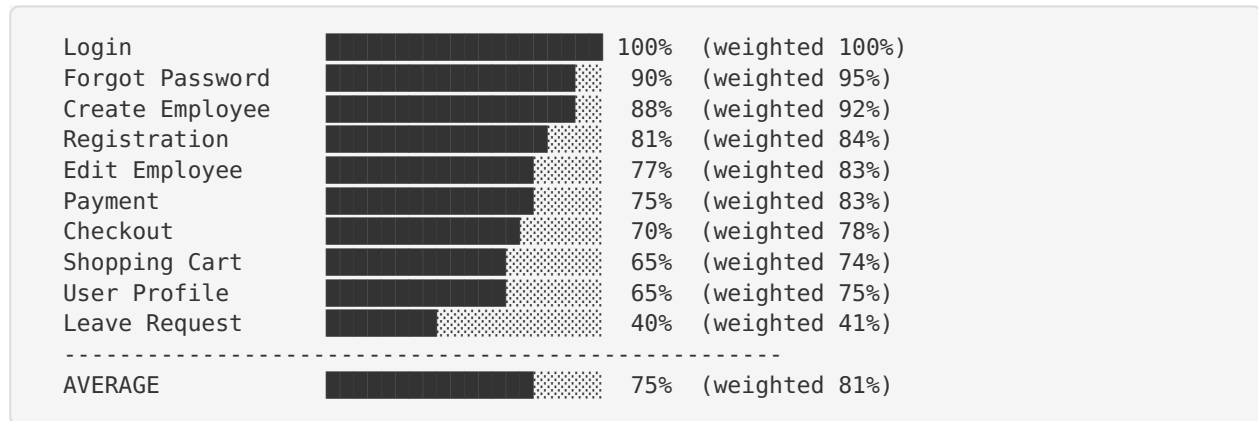
The honest limitation (read this)

This environment has no live LLM and no database, so it cannot produce the real generated suite. Any score claiming to grade live output would be fabricated.

So the CLI scores the **deterministic planner output** (`planScenarios` , all families + deep) — the **PLANNER CEILING**: the best coverage the model is permitted to reach, because the generation prompt forbids it from adding or dropping scenarios. It is a **ceiling, not the live score**. The live suite will land **at or below** these numbers.

The moment this runs in the app with real generated cases, the same `scoreBenchmark()` produces the true leaderboard — no code change, just a different input.

Planner-ceiling leaderboard (captured from an actual run — NOT the live score)



What the ceiling already tells us (before a single fix):

- **Leave Request is the weakest at 40%** — and it fails on **critical** business rules: “End date before start date rejected” and “Insufficient balance rejected” are both missing. That is a real, senior-QA-visible hole, and it is the obvious Phase-3 target.
- **Authorization and Boundary are the recurring weak categories** across requirements (Checkout, Payment, Leave all score 0% on Authorization; Boundary is 0–50% almost everywhere).
- **Login is a genuine 100%** even at the ceiling — a good control that the benchmark is not simply unsatisfiable.

These are ceilings. The live LLM path will likely score lower, which makes the Authorization/Boundary gaps more urgent, not less.

What was NOT done (deliberately deferred)

- **Phase 3 — Fix one category.** Pick the lowest scorer (Leave Request / Authorization / Boundary), fix generation, re-score, prove the number moved. Not started — that is the next focused unit of work.
- **Phase 4 — Leaderboard dashboard + CI gate** (“a PR that drops any score fails”). The scorer already produces the data; wiring it into CI is a follow-up.
- **Phase 5 — New architecture.** No new engine will be added until a benchmark score proves which one is needed. Every future engine must answer: “which benchmark score does this raise?”

Files

File	Role
<code>scripts/gold-benchmarks.ts</code>	Sealed 10-requirement, category-first gold standard
<code>scripts/coverage-match.ts</code>	Shared token-aware coverage detector (one rule everywhere)
<code>scripts/qa-architect-scorer.ts</code>	Pure scorer + leaderboard + planner-ceiling CLI
<code>tests/unit/qa-architect-scorer.test.ts</code>	5 deterministic tests locking the scoring contract
<code>scripts/test-case-audit.ts</code>	Existing loss-audit harness — now imports the shared matcher (also fixed a pre-existing <code>BusinessRuleModel.statement</code> compile error)

Verification: `tsc --noEmit` clean · scorer tests 5/5 · related suites (generation-quality, scenario-planner-standard-coverage, scenario-builder) 60/60 · audit + coverage-origin scripts run clean.

The bottom line

We can now put a number on any requirement's QA coverage against a sealed senior-QA standard, break it down by category, and watch it move release over release. The measuring stick is built and honest. The next move is to pick the lowest bar — **Leave Request at 40%, missing critical business rules** — and prove we can lift it.